

CMPE 492
VeriFi: A Reputation-Based Platform for
Verifiable Financial Predictions

Arda Saygan
Abdurrahman Arslan
Çetin Tiryaki
Volkan Bora Seki
Yusuf Anıl Yazıcı

Advisor:
Hüseyin Birkan Yılmaz

Contents

1	INTRODUCTION	1
1.1	Broad Impact	1
1.2	Ethical Considerations	2
2	PROJECT DEFINITION AND PLANNING	3
2.1	Project Definition	3
2.2	Project Planning	4
2.2.1	Project Time and Resource Estimation	4
2.2.2	Success Criteria	6
2.2.3	Risk Analysis	6
2.2.4	Team Work	6
3	RELATED WORK	9
4	METHODOLOGY	11
5	REQUIREMENTS SPECIFICATION	13
6	DESIGN	15
6.1	Information Structure	15
6.2	Information Flow	15
6.3	System Design	16
6.4	Authentication Design	16
6.5	User Interface Design	17
7	IMPLEMENTATION AND TESTING	19
7.1	Implementation	19
7.2	Testing	20
7.3	Deployment	21
8	RESULTS	22

9	CONCLUSION	23
A	API REFERENCE	25

Chapter 1

INTRODUCTION

1.1 Broad Impact

The financial media landscape is plagued by unverifiable claims. Influencers, analysts, and self-proclaimed experts routinely make bold financial predictions on social media with no accountability mechanism. When predictions prove correct they are amplified; when wrong, they are quietly deleted or ignored. This asymmetry erodes public trust and enables misinformation to thrive in financial discourse.

VeriFi addresses this problem by building a social media platform where every financial prediction is cryptographically signed at the time of posting. Because the signature is generated from the author's private key and bound to the claim content, a prediction cannot be retroactively modified or plausibly denied — even after the original post is deleted. This property, *non-repudiation*, is the core differentiator of VeriFi compared to existing financial social platforms.

The platform benefits several stakeholder groups. Retail investors gain access to a transparent, tamper-proof track record of financial influencers, enabling more informed decisions about whose predictions to follow. Skilled predictors accumulate a credible, verifiable reputation that serves as a professional credential. Researchers and journalists studying financial misinformation gain a public dataset of signed, timestamped predictions with resolution outcomes.

At a societal level, VeriFi promotes financial literacy by shifting discourse toward verifiable, quantified claims and away from vague, unfalsifiable commentary. By making prediction accuracy publicly auditable, the platform creates an economic incentive for quality analysis and a natural disincentive for low-effort, clickbait content.

1.2 Ethical Considerations

Several ethical dimensions require careful attention in the design and operation of VeriFi.

Not investment advice. VeriFi is a reputation-tracking platform, not a financial advisory service. All content is user-generated and not endorsed by the platform operators. The platform displays a prominent disclaimer that no content constitutes investment advice, in compliance with applicable financial regulations.

Privacy and data protection. VeriFi deliberately avoids collecting personally identifiable information (PII). Users are identified solely by their Ethereum wallet address, a pseudonymous identifier derived from a cryptographic key pair. No names, email addresses, phone numbers, or other PII are stored. Post text is deletable by the user to accommodate GDPR and KVKK (Turkish Personal Data Protection Law) rights to erasure. However, the structured claim payload — the financially verifiable component of a prediction — persists even after post deletion. This design reflects the principle that while personal commentary may be retracted, quantified public financial claims carry a different epistemic status analogous to a published prediction.

AI limitations. The planned LLM-based claim extraction feature assists users in structuring natural-language posts into verifiable claims. The system is designed so that the LLM serves as an assistant, not a decision-maker: every extracted claim is presented to the user for explicit confirmation, editing, or rejection before publication. The LLM never publishes claims autonomously. The system also flags when a prediction lacks a measurable target or timeframe, prompting the user to provide those fields rather than inferring them, in order to maintain verifiability.

Sybil resistance. A reputation system is only as trustworthy as its identity layer. By anchoring identity to a cryptographic key pair rather than a platform-controlled account, VeriFi ensures that identity manipulation requires actual cryptographic key control. The challenge-response authentication scheme prevents replay attacks. Planned features such as stake-weighted voting for soft claims further raise the economic cost of Sybil attacks.

Fairness in resolution. Claim resolution in v1 is performed by administrators identified by a whitelisted Ethereum address. Automated oracle-based resolution is planned for future milestones. The platform is designed so that resolved claim status cannot revert, and all resolution actions are tied to the admin’s cryptographic identity, making resolution decisions auditable.

Chapter 2

PROJECT DEFINITION AND PLANNING

2.1 Project Definition

VeriFi is a web-based social media platform for verifiable financial predictions. The central concept is the *Hard Claim*: a quantified, time-bounded financial prediction with the structured fields `{asset, direction, percentage, until}`. Hard claims are cryptographically signed by their author at publication time. When the deadline passes, each claim is resolved as confirmed or rejected by comparing the actual asset price movement against the prediction, either automatically via a financial data oracle or manually by an administrator.

Each user accumulates a *Truth Score*, a global reputation metric derived from the accuracy and difficulty of their resolved claims. The Truth Score is publicly visible, non-editable, and tamper-evident because it is anchored to cryptographically signed claim records.

Authentication is passwordless and wallet-based. Users either generate a native VeriFi wallet (12-word BIP39 mnemonic [3], secp256k1 key derivation via BIP32 [8], private key stored encrypted in browser `localStorage`) or connect an existing MetaMask wallet. Login uses an EIP-191 `personal_sign` challenge-response flow [5]: the server issues a single-use nonce, the client signs it with the private key, and the server recovers the signing address for verification. No password is ever stored or transmitted.

The platform is built as a monorepo with a Django REST Framework backend (deployed on Render) and a Vite/React/TypeScript frontend (deployed on Vercel). The database is SQLite in development and PostgreSQL in production.

In scope (v1):

- Wallet-based authentication (native BIP39 wallet and MetaMask)
- Hard claim creation, feed, and user profiles
- Claim lifecycle management (`undetermined` $\rightarrow$ `confirmed` / `rejected`)
- Manual claim resolution by admin
- Cryptographic proof generation and download
- Standalone proof verification page (no login required)
- Truth Score computation and public display

Out of scope (v1):

- Soft claims and community voting
- Automated oracle-based claim resolution (planned Milestone 3)
- LLM-based automatic claim extraction (stub exists; full feature in progress)
- Domain-specific reputation scores
- Monetization and subscription tiers
- Mobile applications
- Blockchain anchoring / on-chain timestamping

2.2 Project Planning

2.2.1 Project Time and Resource Estimation

The project follows a milestone-driven development schedule aligned with the course calendar. Four milestones and a final senior presentation have been defined.

The team consists of five members, each contributing across frontend and backend work. External resource dependencies include financial data APIs (CoinGecko, Yahoo Finance, Alpha Vantage) for oracle-based resolution, and a locally hosted language model (via LM Studio) for LLM claim extraction.

Table 2.1: Project milestones and their key deliverables.

Milestone	Date	Deliverables
M1	March 8, 2026	Authentication and identity: native wallet (BIP39/secp256k1), MetaMask integration, challenge-response JWT login, unified login UI
M2	April 5, 2026	Core social and claim pipeline: post composer, LLM claim extraction (Agno), claim card UI, signed proof packages, public social feed
M3	May 17, 2026	Resolution, reputation and verification: oracle integration (CoinGecko, Yahoo Finance), Truth Score, proof verifier page, feed filtering by asset
Final	June 11, 2026	End-to-end demo, UI/UX polish, edge case handling, performance pass, final documentation

2.2.2 Success Criteria

The project will be considered successful if the following criteria are met by the final presentation:

- A registered user can create a hard claim; the claim is cryptographically signed and stored.
- A visitor can browse the public claim feed without registering.
- An expired claim can be resolved (confirmed or rejected) and the author's Truth Score is updated accordingly.
- Any third party can download a proof file and verify it on the standalone proof verification page without logging in.
- The system correctly rejects tampered or forged proof files.
- The feed page loads within 2 seconds under typical usage conditions.
- No critical security vulnerabilities exist in the authentication or claim submission flow.

2.2.3 Risk Analysis

2.2.4 Team Work

The project team consists of five members. Responsibilities are distributed as follows:

The team holds weekly advisory meetings on Tuesdays (17:30–18:00) with the project advisor. Meeting notes and decisions are recorded in the project GitHub wiki. Rush weekends with concentrated development effort have been scheduled around milestone deadlines.

Table 2.2: Identified risks, likelihood, impact, and mitigation strategies.

Risk	Likelihood	Impact	Mitigation
Oracle API down-time or rate limiting	Medium	High	Exponential backoff retries; fallback to secondary data source; manual admin resolution as last resort
LLM extraction quality insufficient	High	Medium	v1 ships with manual structured entry form; LLM extraction is an enhancement, not a blocker
Truth Score formula disputes	Medium	Medium	Formula publicly documented and fixed at v1 launch; transparent claim records allow independent verification
Browser <code>localStorage</code> key loss	Medium	High	Mnemonic displayed and backup strongly encouraged at registration; no server-side key recovery by design
Private key leakage from <code>localStorage</code>	Low	Critical	AES-256-GCM encryption with PBKDF2-SHA256 (100,000 iterations); key never transmitted to server
GDPR / KVKK compliance	Low	High	No PII collected; post text deletable; claim payloads treated as public reputation data

Table 2.3: Team member responsibilities.

Member	Primary Responsibilities
Arda Saygan	Backend architecture; hard claim data models and API endpoints; project repository management
Abdurrahman Arslan	MetaMask integration; frontend PostCard and claim card UI components; requirements specification
Çetin Tiryaki	LLM experimentation and benchmarking; claim extraction UI; report contributions
Volkan Bora Seki	AI/LLM infrastructure (Agno + LM Studio); multilingual claim extraction; LLM benchmark design
Yusuf Anıl Yazıcı	Native wallet and key management; cryptographic authentication frontend; midterm report; deployment

Chapter 3

RELATED WORK

The problem of tracking and verifying financial predictions has been approached from several directions in both industry and academia.

Prediction Markets. Prediction markets are exchange-traded platforms where users buy and sell contracts on future events, with prices reflecting aggregate probability estimates [7]. Platforms such as Polymarket and PredictIt allow users to stake real money on event outcomes, providing strong accountability: incorrect predictions result in direct financial loss. While effective for accountability, these systems are regulated as financial instruments in many jurisdictions and do not support the social media dynamics — posts, feeds, followers, profiles — that VeriFi targets. Augur [4] extends the model to a decentralised, blockchain-based architecture, but requires on-chain interaction and cryptocurrency holdings, creating a high barrier for casual users.

Social Trading Platforms. eToro and similar platforms allow users to publish and copy investment portfolios, providing a form of reputation tracking based on actual trade performance. However, these platforms require linking real brokerage accounts, making them inaccessible to users who wish to share predictions without executing trades. StockTwits provides a Twitter-like social layer for financial commentary but lacks any systematic verification or scoring mechanism: predictions and outcomes are not tracked.

Reputation Systems. Reputation systems in online communities, such as Stack Overflow’s karma system or academic citation indices, have been studied extensively [2]. A key finding is that reputation scores are only as trustworthy as the underlying identity system and the non-repudiability of the actions they measure. VeriFi applies this insight to financial predictions by anchoring reputation to cryptographic identity and signed claim records.

Cryptographic Non-Repudiation. The use of digital signatures for non-repudiation of electronic documents is well established [1]. Ethereum’s

EIP-191 `personal_sign` standard [5] extends this to wallet-based signatures, which VeriFi uses as the foundation of its proof system. The key contribution of VeriFi is applying this mechanism to social media content — specifically financial predictions — so that deletion or denial of a claim becomes cryptographically impossible for anyone who has saved the signed proof.

LLM-Assisted Structured Extraction. Recent work on large language model-based information extraction has demonstrated strong performance on converting natural language into structured records [6]. VeriFi uses an LLM (via the Agno framework with a locally hosted model) to assist users in extracting structured `{asset, direction, percentage, timeframe}` tuples from free-form text. A critical design constraint, derived from team discussions, is that the LLM must not infer missing fields — especially timeframes — but must prompt the user to supply them explicitly, preserving the verifiability of every published claim.

Chapter 4

METHODOLOGY

VeriFi is built as a monorepo web application with clearly separated frontend and backend layers.

Backend. The backend is a REST API built with Django 6 and Django REST Framework (DRF). Authentication uses `djangorestframework-simplejwt` for JWT issuance and validation, and the `eth-account` library for Ethereum signature verification. The backend is stateless: no server-side sessions are maintained. All authentication state is encoded in the JWT token. The database is SQLite in development and PostgreSQL in production (Render).

Frontend. The frontend is a React 19 + TypeScript single-page application (SPA) built with Vite 7. UI components use Tailwind CSS v4 and the `shadcn/ui` component library. Cryptographic operations (BIP39 mnemonic generation, BIP32 key derivation, secp256k1 signing, AES-256-GCM key encryption) are performed entirely client-side using `@scure/bip39`, `@scure/bip32`, `@noble/secp256k1`, and `@noble/hashes` — pure TypeScript implementations with no native dependencies. The frontend is deployed on Vercel.

Authentication. The authentication system is passwordless and wallet-based. Two wallet modes are supported: (1) a native VeriFi wallet based on BIP39 mnemonic generation and BIP32/BIP44 key derivation, with the private key stored AES-256-GCM encrypted in browser `localStorage`; and (2) MetaMask via the EIP-1193 browser extension API. Both modes use an identical EIP-191 `personal_sign` challenge-response flow for login, described in full in Section 6.4.

Claim Signing. Every hard claim is cryptographically signed with the author’s secp256k1 private key at creation time. The signature covers a canonical JSON serialisation of the claim payload: `{author_address, asset, direction, percentage, until, server_timestamp}`. This produces a proof bundle that allows any third party to verify — using only the

standard Ethereum public-key infrastructure — that a specific user made a specific prediction at a specific time.

LLM Claim Extraction. The system includes a claim extraction endpoint (`POST /api/posts/extract-claims/`) which, in the planned implementation, will pass post content to an LLM (Agno framework with a locally hosted language model) and return a structured list of detected claims. The LLM output schema is `{claims: [{asset, direction, target_value, target_unit, timeframe_iso, language}]}`. Each extracted claim is presented to the user as a Claim Card for explicit confirmation, editing, or rejection before publication. The LLM was benchmarked on a custom dataset of English and Turkish prediction sentences spanning single-claim, multi-claim, and edge-case inputs. As of the midterm, the endpoint returns an empty list (stub); the full implementation is in progress.

Oracle-Based Resolution. Claim resolution will be automated by a scheduled backend job that, upon claim expiry, queries a financial data oracle (CoinGecko for cryptocurrency assets; Yahoo Finance and Alpha Vantage for stocks, forex, and commodities) to determine whether the price movement matched the prediction. The resolution criterion is: did the asset move at least `percentage%` in the predicted `direction` by the `until` date? Resolution status cannot revert once set, and the resolution is idempotent. This feature is targeted for Milestone 3.

Truth Score. Each user’s Truth Score is a single global numeric reputation metric updated automatically on every claim resolution. The formula is designed to weight claim difficulty: harder predictions (wider percentage targets, longer timeframes, more volatile assets) yield greater reputation gains when confirmed and greater penalties when rejected. The exact weighted formula is under active design as an open question (OQ-01 in the project SRS).

Chapter 5

REQUIREMENTS SPECIFICATION

The following use cases define the primary interactions with the system.

UC-1: Register with Native Wallet. A new user opens the registration page. The system generates a 12-word BIP39 mnemonic and displays it prominently with instructions to record it. The user sets an encryption passphrase. The system derives a secp256k1 private key via BIP32/BIP44 path `m/44'/60'/0'/0/0`, derives an Ethereum address (Keccak-256 of uncompressed public key, last 20 bytes), encrypts the private key with AES-256-GCM + PBKDF2-SHA256, stores the ciphertext in browser `localStorage`, and registers the address with the backend. The backend issues a JWT. The user is now logged in.

UC-2: Login with Native Wallet. A returning user enters their passphrase. The system decrypts the stored private key. A single-use nonce (5-minute TTL) is fetched from the backend. The system signs the nonce using EIP-191 `personal_sign` and sends `{address, nonce, signature}` to the backend. The backend recovers the signing address from the signature and verifies it matches the registered address. A new JWT is issued.

UC-3: Login with MetaMask. A user with MetaMask installed clicks the MetaMask login option. The system calls `eth_requestAccounts` to retrieve the user's address. If not yet registered, the address is registered automatically. A challenge nonce is fetched and signed via MetaMask's `personal_sign` popup. The backend flow is identical to UC-2.

UC-4: Create a Hard Claim. An authenticated user fills out the hard claim form: claim text (natural language), asset (dropdown from the asset API), direction (Bullish or Bearish), expected percentage move (0–100), and deadline (a future date). The system validates all fields server-side and stores the claim with status `undetermined`, linked to the user's wallet address.

UC-5: Browse the Public Feed. Any user (authenticated or not) opens the feed page. The system loads all hard claims sorted by creation time, newest first. Each PostCard displays: author address, asset, direction, percentage, deadline, and current status badge. The feed is accessible without login.

UC-6: Resolve a Hard Claim (Admin). An administrator (identified by a whitelisted Ethereum address in `ADMIN_ADDRESSES`) sends a `PATCH` request to `/api/posts/hard-claims/{id}/update-status/` with the new status (confirmed or rejected). The backend verifies the caller’s JWT address is in the admin whitelist. The status is updated and cannot revert.

UC-7: Download and Verify a Proof. An authenticated user views a confirmed hard claim and clicks “Download Proof.” The system generates and downloads a JSON file containing `{claim_payload, signature, wallet_address, server_timestamp}`. Any user can later upload this file to the standalone proof verification page. The verifier reconstructs the signed payload, recovers the signing address from the signature using `eth_account.recover_message()`, and confirms it matches `wallet_address`.

UC-8: Non-Repudiation of Deleted Post. A user saves the signed proof of another user’s hard claim. The claim author later deletes their post (exercising GDPR / KVKK rights). The post text is removed, but the claim payload record persists in the database. The saved proof remains cryptographically valid. The proof verification page confirms the signature, providing tamper-evident evidence that the prediction was made, regardless of post deletion.

UC-9: View User Profile. Any visitor navigates to a user’s profile page by wallet address. The system displays the user’s Truth Score, full claim history with resolution outcomes, and proof download links for confirmed claims. No login is required.

UC-10: LLM-Assisted Claim Extraction. An authenticated user writes a natural-language post containing one or more financial predictions. The system submits the post content to the LLM extraction endpoint. The LLM identifies claim candidates and returns structured `{asset, direction, percentage, timeframe}` tuples. If a required field (particularly the timeframe) is missing, the system prompts the user to provide it explicitly rather than inferring it. The user confirms, edits, or rejects each Claim Card before publication.

Chapter 6

DESIGN

6.1 Information Structure

The core data models and their relationships are described below.

WalletUser represents a registered user. Fields: `id` (auto), `address` (42-character Ethereum address, unique, stored lowercase), `created_at`.

Asset represents a tradable financial asset. Fields: `id`, `name` (e.g., “Bitcoin”), `symbol` (e.g., “BTC”), `description` (optional text).

HardClaim is the central entity representing a verifiable financial prediction. Fields: `id`, `author` (FK to `WalletUser`), `text` (natural-language description), `asset` (FK to `Asset`), `direction` (“Bullish” or “Bearish”), `percentage` (float, 0–100), `until` (future date, enforced by DB constraint: `until > created_at`), `created_at`, `status` (enumeration: `undetermined` / `confirmed` / `rejected`).

Post (legacy) is a free-form social post retained for backward compatibility. Fields: `id`, `author` (FK to `WalletUser`, CASCADE delete), `content` (max 500 characters), `created_at`.

The **HardClaim** model is the canonical claim structure. The legacy **Post/Claim** models will be deprecated in a future milestone.

6.2 Information Flow

Authentication flow (native wallet): The browser generates a mnemonic and derives a `secp256k1` key pair entirely client-side. The address is registered with the backend via `POST /api/auth/register/`. On login, the browser requests a nonce from `GET /api/auth/challenge/`, signs it with the private key (EIP-191), and sends the signature to `POST /api/auth/login/`. The

backend uses `eth_account.recover_message()` to recover the signing address and issues a JWT on match.

Claim creation flow: An authenticated user submits the hard claim form. The frontend sends `POST /api/posts/hard-claims/` with a Bearer JWT header. The backend validates the JWT, extracts the wallet address as the claim author, validates all claim fields server-side, and stores the record with status `undetermined`.

Claim resolution flow: After a claim's `until` date passes, an admin (or, in future milestones, an automated oracle job) submits `PATCH /api/posts/hard-claims/{id}/update/`. The backend verifies admin identity, updates status, and triggers Truth Score recalculation for the claim author.

6.3 System Design

The system follows a strict client-server separation. The frontend is a static SPA deployed on Vercel that communicates with the backend exclusively through the REST API over HTTPS. The backend is stateless: all authenticated state is in the JWT token carried by the client. The private key never leaves the browser.

The REST API is organized into two Django apps:

- `accounts` — authentication endpoints: `/api/auth/register/`, `/api/auth/challenge/`, `/api/auth/login/`
- `posts` — content endpoints: `/api/posts/`, `/api/posts/hard-claims/`, `/api/posts/assets/`, `/api/posts/extract-claims/`

Cross-origin requests from the frontend origin (Vercel domain and `localhost:5173` in development) are handled by `django-cors-headers`.

6.4 Authentication Design

The authentication system is designed around the invariant that the private key is never transmitted to or stored by the server. The server only sees a wallet address (a 20-byte Keccak-256 hash of a public key) and a cryptographic signature over a server-issued nonce.

The native VeriFi wallet uses BIP39 (12-word mnemonic, 128 bits of entropy), BIP32/BIP44 key derivation (path `m/44'/60'/0'/0/0`), and standard Ethereum address derivation. The private key is encrypted with AES-256-GCM using a key derived from the user's passphrase via PBKDF2-

SHA256 (100,000 iterations) and stored as a JSON ciphertext object in browser `localStorage`.

Login uses EIP-191 `personal_sign`: the server generates a random hex nonce (32 bytes, 5-minute TTL, single-use), the client signs it, and the server uses `eth_account.recover_message()` to verify the signature. The following table summarises the security properties of both wallet modes.

Table 6.1: Security properties of native wallet vs. MetaMask authentication.

Property	Native Wallet	MetaMask
Private key location	<code>localStorage</code> (AES-256-GCM encrypted)	MetaMask encrypted v
Private key transmitted?	Never	Never
Replay attack protection	Single-use nonce (5-min TTL)	Single-use nonce (5-min
Signature scheme	EIP-191 <code>personal_sign</code> (secp256k1)	EIP-191 <code>personal_sign</code>
Session mechanism	Stateless JWT (7-day expiry)	Stateless JWT (7-day e

6.5 User Interface Design

The frontend implements the following main views:

- **Login / Registration Page:** Wallet generation wizard, mnemonic display, passphrase setup, and MetaMask connect button.
- **Feed Page:** Public list of hard claims rendered as PostCards, each showing author address, asset, direction, percentage, deadline, and status badge. New Claim dialog accessible to authenticated users.
- **Post Detail Page:** Expanded view of a single post with all associated hard claims and proof download options.
- **Profile Page:** User’s wallet address (with 60-second timed private key reveal, partially obfuscated), Truth Score, and complete claim history.
- **Proof Verification Page:** Standalone page (no login required) for uploading and verifying a downloaded proof JSON file.
- **Post Creation Page:** Free-form text composer with optional LLM-assisted claim extraction.

- **Claim Review Page:** Displays extracted Claim Cards for user confirmation before publishing.

The design language uses Tailwind CSS v4 utility classes and the shadcn/ui component library for consistent, accessible UI components (buttons, dialogs, badges, alerts, selects, inputs, textareas).

Chapter 7

IMPLEMENTATION AND TESTING

7.1 Implementation

As of the midterm (March 2026), the following features are implemented and functional.

Authentication system:

- Native wallet: BIP39 mnemonic generation, secp256k1 key derivation, passphrase-based AES-256-GCM encryption, `localStorage` storage, challenge-response JWT login.
- MetaMask: EIP-1193 `eth_requestAccounts` and `personal_sign` integration using the same backend challenge-response flow.
- Unified login UI supporting both wallet modes on a single page.
- JWT-based session management (7-day token lifetime, stateless).
- Logout: clears JWT, wallet address, and encrypted key from `localStorage`.

Hard claims:

- Hard claim creation form with asset dropdown, direction selector, percentage input, and date picker.
- Backend validation: percentage range (0–100), future deadline enforcement (DB-level constraint), authenticated author linking.
- Public hard claim feed rendered as PostCard components.

- Hard claim filtering by author address (`?address=` query parameter).
- Admin-only manual claim status update endpoint.
- Asset list API with pre-seeded assets (Bitcoin, Ethereum, US Dollar, and others).

User interface:

- Feed page with PostCard components showing all hard claim fields and status badges.
- Profile page with timed private key reveal (60-second auto-hide, partial obfuscation, copy-to-clipboard).
- Responsive layout with navbar, Tailwind v4 theming, and shadcn/ui components.

Pending for upcoming milestones:

- Cryptographic proof signing, proof download UI, and standalone proof verification page (Milestone 2).
- Truth Score model, calculation formula, and public display (Milestone 2/3).
- Automated oracle-based claim resolution with exponential-backoff retries (Milestone 3).
- LLM claim extraction (currently a stub returning `[]`); full implementation in progress.
- Feed filtering by asset (Milestone 3).
- Post text deletion for GDPR / KVKK compliance.

7.2 Testing

Security testing. An XSS vulnerability scan was conducted on 2026-03-17 using XSSStrike v3.1.5 targeting both the frontend (`localhost:5173`) and backend (`localhost:8000`) at commit `861894b`. All 28 fuzzer payloads — including `<script>`, `<svg>`, and event handler injections (`onload`, `onclick`) — were blocked. No XSS vulnerabilities were detected. React’s automatic content escaping (no `dangerouslySetInnerHTML` usage) and Django REST Framework’s JSON-only response format both contribute to this result.

Manual testing. Authentication flows (native wallet and MetaMask), hard claim creation, and feed display have been tested manually in the development environment. The challenge-response authentication flow has been verified end-to-end: nonce issuance, EIP-191 signing in the browser, server-side `eth_account` signature recovery, and JWT issuance.

LLM benchmark. A benchmark dataset has been designed to evaluate LLM claim extraction quality. The dataset contains examples in English and Turkish across four categories: (1) single-claim factual predictions, (2) multi-claim sentences, (3) combined quantitative predictions, and (4) noise / near-miss inputs that should not produce claims (e.g., intent statements, non-quantified commentary). Results will be reported in the final submission.

Automated unit and integration testing with a structured test suite are planned for the upcoming milestones.

7.3 Deployment

The platform is deployed to production:

- **Frontend:** Vercel, building from the `frontend/` directory with `pnpm build`. Configuration in `frontend/vercel.json`. SPA routing handled via `rewrites` config.
- **Backend:** Render, running `gunicorn` as the WSGI server. Configuration in `backend/render.yaml`. Environment variables (`SECRET_KEY`, `DATABASE_URL`, `ADMIN_ADDRESSES`, `CORS origins`) managed through Render’s dashboard.
- **Database:** SQLite in development; PostgreSQL managed by Render in production.

CORS is configured to permit requests from the Vercel frontend domain in production and from `localhost:5173` during development.

Chapter 8

RESULTS

As of the midterm submission, the authentication system and core hard claim pipeline are fully operational. Users can register and log in with both a native VeriFi wallet and MetaMask, create hard claims through a structured form, and browse the public claim feed. The admin claim resolution mechanism is functional, enabling manual lifecycle management from **undetermined** to **confirmed** or **rejected**. The platform is live in production on Vercel (frontend) and Render (backend).

Milestone 1 (March 8, 2026) was completed on schedule, delivering the full authentication system. The hard claim data model, feed, and basic profile pages were delivered as part of subsequent sprint work ahead of the midterm.

The LLM claim extraction feature is partially implemented. The extraction endpoint exists and is reachable; the LLM benchmark dataset has been designed and populated with English and Turkish examples. Full integration with the Agno framework and a hosted language model is in progress and targeted for Milestone 2.

The cryptographic proof system (signing at claim creation, proof download, and standalone verification) and the Truth Score calculation are the two most significant remaining features of the platform and are targeted for Milestones 2 and 3 respectively.

Chapter 9

CONCLUSION

VeriFi aims to bring cryptographic accountability to financial social media — a domain currently dominated by unverifiable claims and selective recall. By anchoring every prediction to a signed proof tied to a cryptographic wallet identity, the platform makes financial predictions permanently attributable and verifiable, even after the original post is deleted.

At the midterm stage, the core infrastructure is in place: a full wallet-based authentication system supporting both native BIP39 wallets and MetaMask, the hard claim creation and feed pipeline, manual admin resolution, and a live production deployment. The authentication flow represents a significant technical achievement, implementing the full BIP39/BIP32/secp256k1/EIP-191 stack client-side with AES-256-GCM key protection, while maintaining full compatibility with the Ethereum ecosystem.

The remaining work focuses on completing the reputation and proof systems: cryptographic proof signing and download, the standalone verifier page, oracle-based automated resolution, and the Truth Score formula and calculation engine. These features constitute the distinctive value proposition of VeriFi and will be the primary development focus for the second half of the semester.

Bibliography

- [1] Don Johnson, Alfred Menezes, and Scott Vanstone. The elliptic curve digital signature algorithm (ECDSA). *International Journal of Information Security*, 1(1):36–63, 2001.
- [2] Audun Jøsang, Roslan Ismail, and Colin Boyd. A survey of trust and reputation systems for online service provision. *Decision Support Systems*, 43(2):618–644, 2007.
- [3] Marek Palatinus, Pavol Rusnak, Aaron Voisine, and Sean Bowe. BIP-39: Mnemonic code for generating deterministic keys. Bitcoin Improvement Proposals, 2013. Available at <https://github.com/bitcoin/bips/blob/master/bip-0039.mediawiki>.
- [4] Jack Peterson, Joseph Krug, Micah Zoltu, Austin Williams, and Stephanie Alexander. Augur: a decentralized oracle and prediction market platform. Technical report, Forecast Foundation, 2019. Version 2.0. Available at <https://github.com/AugurProject/whitepaper>.
- [5] Geth Team. EIP-191: Signed data standard. Ethereum Improvement Proposals, 2016. Available at <https://eips.ethereum.org/EIPS/eip-191>.
- [6] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, 2022.
- [7] Justin Wolfers and Eric Zitzewitz. Prediction markets. *Journal of Economic Perspectives*, 18(2):107–126, 2004.
- [8] Pieter Wuille. BIP-32: Hierarchical deterministic wallets. Bitcoin Improvement Proposals, 2012. Available at <https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki>.

Appendix A

API REFERENCE

Table A.1: Full REST API endpoint reference.

Method	Path	Auth	Description
POST	/api/auth/register/	None	Register a wa returns JWT
GET	/api/auth/challenge/	None	Issue a singl for signing
POST	/api/auth/login/	None	Verify EIP-19 issue JWT
GET	/api/posts/	None	List all posts
POST	/api/posts/	Bearer	Create a new
POST	/api/posts/extract-claims/	Bearer	LLM claim (stub)
GET	/api/posts/hard-claims/	None	List hard cl ?address=)
POST	/api/posts/hard-claims/	Bearer	Create a new
PATCH	/api/posts/hard-claims/{id}/update-status/	Bearer (Admin)	Resolve a cla
GET	/api/posts/assets/	None	List availab assets